

Architectural Reflection: Lab 4 — Decorator Pattern

Course: Integrative Programming and Technologies (IPT)

Module: Structural Design Patterns in Web Application Development

Topic: HTTP Middleware Pipeline (Dynamic Responsibility Attachment)

1. Spotting the Smell: Middleware Subclass Explosion

In `without_web.php`, cross-cutting concerns (Logging, Caching, Retry, Token Counting) are added using class inheritance:

```
class LoggingHttpClient extends BaseHttpClient { ... }
```

```
class CachingHttpClient extends BaseHttpClient { ... }
```

```
class LoggingCachingHttpClient extends LoggingHttpClient { ... }
```

This design leads to class explosion:

1. **Exponential Class Bloat (2^n):** 4 features require $2^4 = 16$ subclasses to cover every combination. Adding a 5th feature requires 32 classes.
 2. **Code Duplication:** Because PHP lacks multiple inheritance, combination classes like `LoggingCachingHttpClient` must manually duplicate caching and logging logic.
 3. **Static Composition:** Features cannot be toggled or dynamically re-ordered at runtime based on environment, user permissions, or request parameters.
-

2. Architectural Fix: The GoF Decorator Pattern

The refactored implementation (`with_pattern.php`) applies the **Decorator Pattern**:

- **Component Interface:** `HttpClient` with `get(string $url): string`.
- **Concrete Component:** `BaseHttpClient` performs the fundamental HTTP network call.
- **Decorator Base:** `abstract class HttpDecorator implements HttpClient` contains a wrapped `HttpClient` reference via composition.

- **Concrete Decorators:** `LoggingDecorator`, `CachingDecorator`, `RetryDecorator`, and `TokenCounterDecorator` augment behavior before or after delegating to `$this->wrapped->get($url)`.

Clients can stack any combination of decorators at runtime:

```
$client = new TokenCounterDecorator(new RetryDecorator(new CachingDecorator(new
LoggingDecorator(new BaseHttpClient()))));
```

All 16 combinations are achieved with only 5 classes ($N + 1$ instead of 2^N).

3. Academic Grounding (Gamma et al., 1994)

"Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."

— Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (*Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, p. 175)

Gamma et al. emphasize that decorators provide a pay-as-you-go approach to feature addition. Instead of predicting every combination in a rigid inheritance tree, responsibility is added incrementally by wrapping objects.

4. Pipeline Order Sensitivity & Practical Web Applications

Wrapping order fundamentally dictates pipeline execution behavior:

- **`Logging(Caching(Base))`**: The logger is outermost. Every request triggers a log line, including cache hits.
- **`Caching(Logging(Base))`**: The cache is outermost. On cache hits, the response returns immediately; the inner logger is never invoked, logging only true network cache misses.

Real-World Web Applications in PHP Frameworks

- **HTTP Middleware (PSR-15 / Laravel)**: Pipeline layers (Authentication → Throttle → CORS → Logging → Controller Handler) wrap request-response cycles.
- **Guzzle HTTP Client Handlers**: Guzzle's `HandlerStack` wraps client handlers with middleware for retries, redirects, and request signing.

- **E-Commerce Checkout Calculations:** Dynamically decorating base cart orders with coupons, VAT, regional shipping fees, and gift wrapping.